1. Docker

What is Docker?

Docker is a containerization platform that packages an application along with its code, libraries, dependencies, and runtime into a container.

A container runs the application the same way on every system, avoiding the common problem:

> "It works on my machine but not on yours."

Purpose of Docker

Package applications with dependencies.

Run applications consistently across environments.

Faster deployment.

Lightweight compared to Virtual Machines.

Easy application sharing.

Real-time Example

Suppose you developed a Python web application.

Without Docker:

Install Python

Install Flask

Install required libraries

Configure environment

With Docker:

Build one Docker image.

Run it anywhere using Docker.

Everything is already inside the container.

Docker Components

Docker Engine

Docker Image

Docker Container

Dockerfile

Docker Hub

Docker Volume

Docker Network

2. Docker Compose

What is Docker Compose?

Docker Compose is a tool used to define and run multiple Docker containers together using a single YAML file (docker-compose.yml).

Instead of starting containers one by one, Compose starts all services together.

Purpose of Docker Compose

Manage multiple containers.

Create networks automatically.

Create volumes automatically.

Easy local development.

One command starts everything.

Real-time Example

A shopping application has:

Frontend

Backend

MySQL Database

Redis Cache

Without Compose:

docker run frontend

docker run backend

docker run mysql

docker run redis

With Compose:

docker compose up

Everything starts automatically.

docker-compose.yml Example

version: "3"

services:

```
 web:
   image: nginx
   ports:
    - "80:80"

 database:
   image: mysql
   environment:
    MYSQL_ROOT_PASSWORD: root123
```

Common Commands

Start:

```
docker compose up
```

Background:

```
docker compose up -d
```

Stop:

```
docker compose down
```

View Containers:

docker compose ps

View Logs:

docker compose logs

3. Kubernetes

What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automatically manages Docker containers across multiple servers.

If Docker creates containers, Kubernetes manages them at scale.

Purpose of Kubernetes

Deploy applications.

Scale applications automatically.

Recover failed containers.

Load balancing.

Rolling updates.

High availability.

Self-healing.

Real-time Example

Netflix has thousands of users.

If one server fails:

Docker alone cannot automatically replace it.

Kubernetes detects the failure and starts a new container automatically.

Users continue using the application without interruption.

Why Kubernetes is Needed

Imagine an e-commerce application with:

100 frontend containers

50 backend containers

10 databases

Managing these manually is very difficult.

Kubernetes automatically:

Deploys containers

Monitors health

Restarts failed containers

Balances traffic

Scales based on demand

Kubernetes Architecture

Control Plane (Master Node)

Responsible for managing the cluster.

Components:

API Server

Scheduler

Controller Manager

etcd

Worker Node

Runs the application containers.

Components:

Kubelet

Container Runtime

Kube Proxy

Pods

Kubernetes Objects

Pod

ReplicaSet

Deployment

Service

Namespace

ConfigMap

Secret

Persistent Volume

Ingress

Docker vs Docker Compose vs Kubernetes

Docker	Docker Compose	Kubernetes
Creates containers	Manages multiple containers	Manages containers across many servers
Single application	Multi-container application	Production orchestration
One container at a time	Multiple containers together	Thousands of containers
Manual scaling	Limited scaling	Automatic scaling
Local development & testing	Local development	Production environments

---

Workflow

Write Application

↓

Create Dockerfile

↓

Build Docker Image

↓

Run Container using Docker

↓

Use Docker Compose (Multiple Containers)

↓

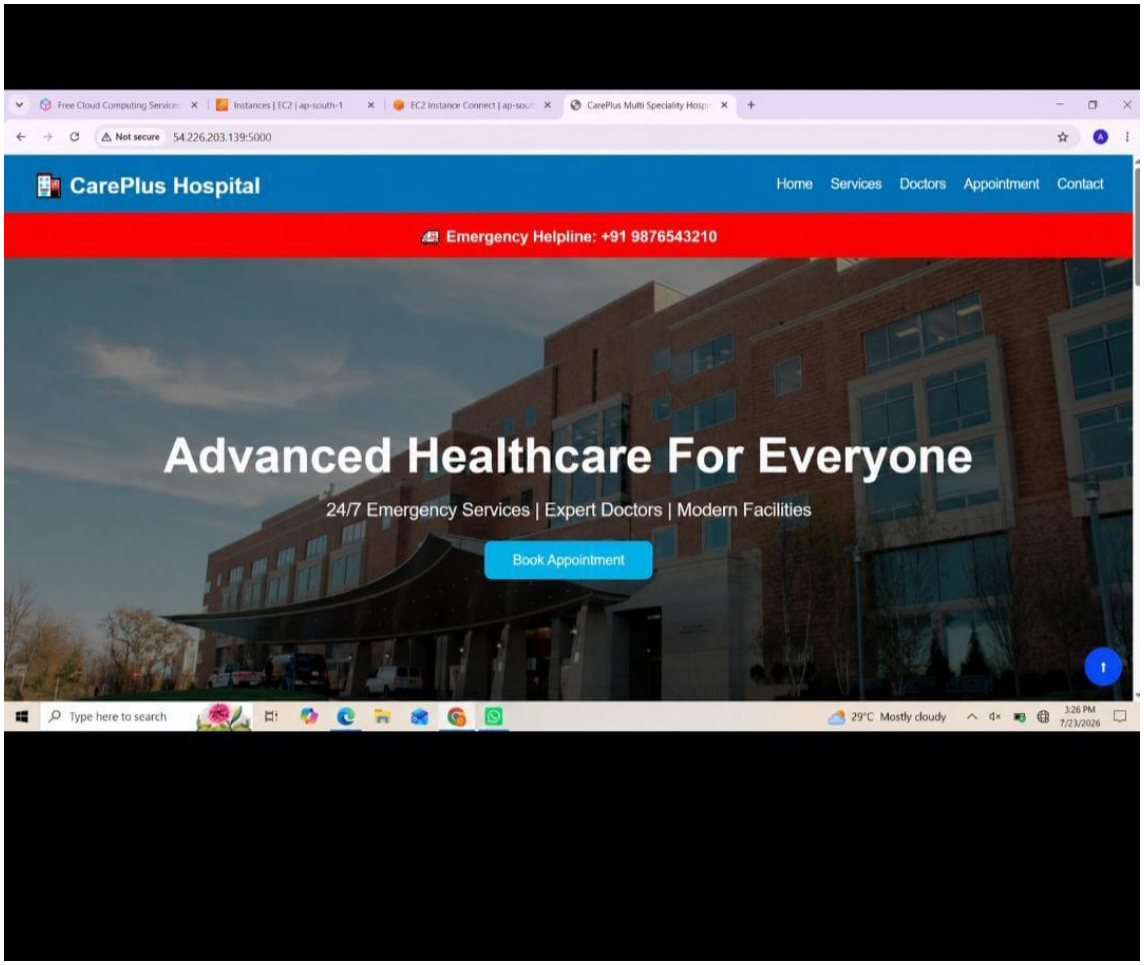
Deploy to Kubernetes Cluster

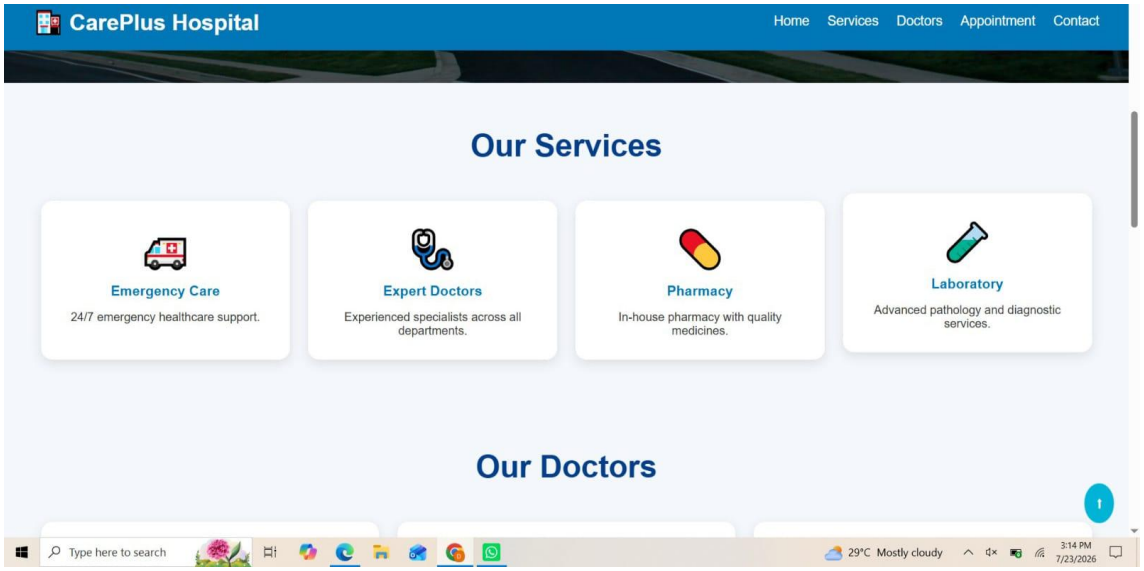
↓

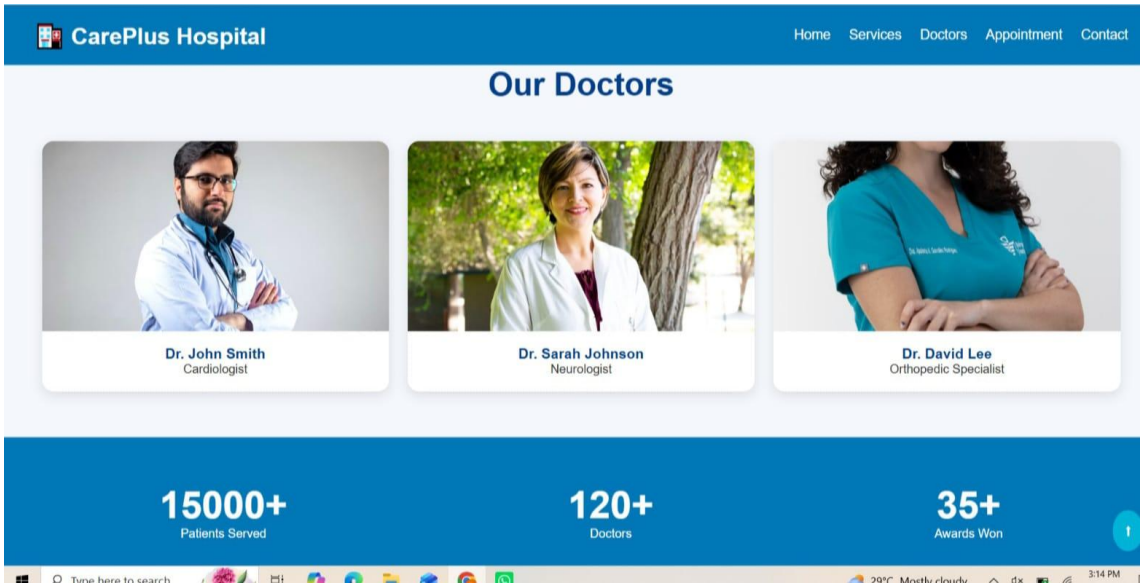
Kubernetes manages scaling, load balancing,

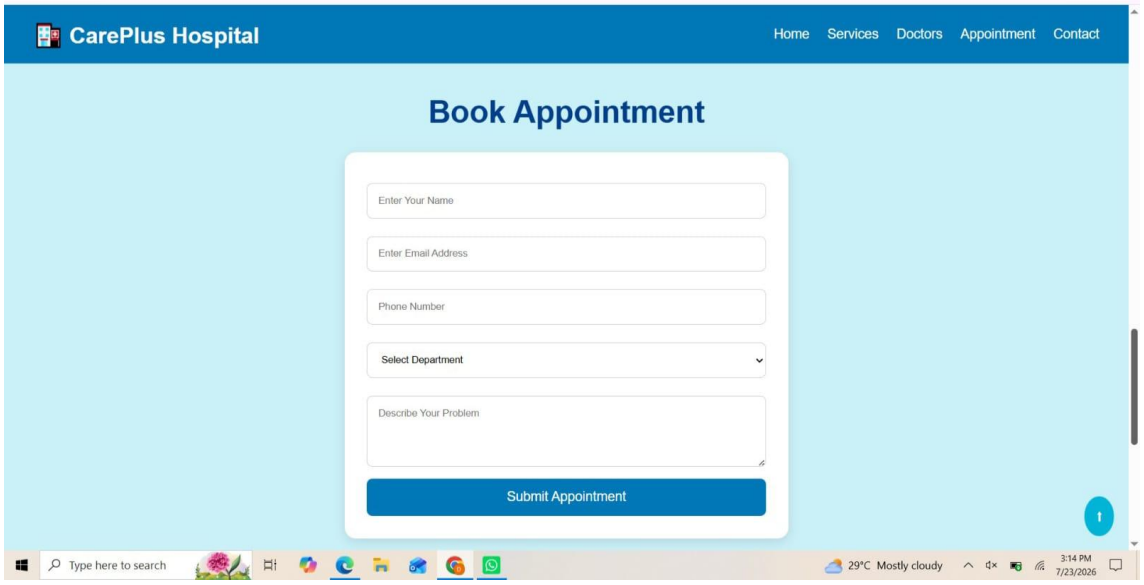
self-healing, and high availability

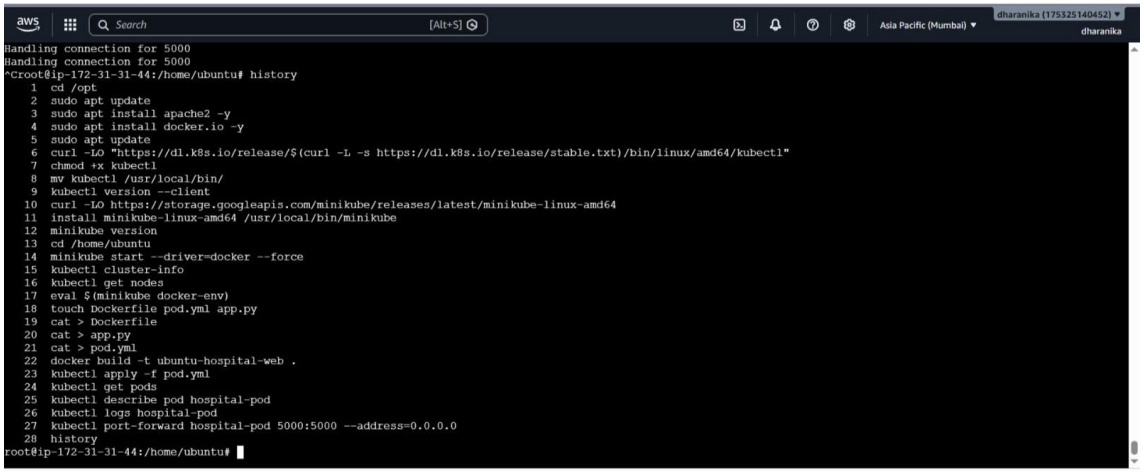
I created a website using this


Not secure 54.226.203.139:5000
CarePlus Hospital
Home Services Doctors Appointment Contact
Emergency Helpline: +91 9876543210
Advanced Healthcare For Everyone
24/7 Emergency Services | Expert Doctors | Modern Facilities
Book Appointment
Type here to search
29°C Mostly cloudy
3:26 PM
7/23/2026


CarePlus Hospital
Home Services Doctors Appointment Contact
Our Services
Emergency Care
24/7 emergency healthcare support.
Expert Doctors
Experienced specialists across all departments.
Pharmacy
In-house pharmacy with quality medicines.
Laboratory
Advanced pathology and diagnostic services.
Our Doctors
Type here to search
29°C Mostly cloudy
3:14 PM
7/23/2026


CarePlus Hospital
Home
Services
Doctors
Appointment
Contact
Our Doctors
Dr. John Smith
Cardiologist
Dr. Sarah Johnson
Neurologist
Dr. David Lee
Orthopedic Specialist
15000+
Patients Served
120+
Doctors
35+
Awards Won


CarePlus Hospital
Home
Services
Doctors
Appointment
Contact
Book Appointment
Enter Your Name
Enter Email Address
Phone Number
Select Department
Describe Your Problem
Submit Appointment
Type here to search
29°C Mostly cloudy
3:14 PM
7/23/2026


aws
Search
[Alt+S]
Asia Pacific (Mumbai)
dharanika (175325140452)
dharanika
Handling connection for 5000
Handling connection for 5000
^Croot@ip-172-31-31-44:/home/ubuntu# history
1 cd /opt
2 sudo apt update
3 sudo apt install apache2 -y
4 sudo apt install docker.io -y
5 sudo apt update
6 curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
7 chmod +x kubectl
8 mv kubectl /usr/local/bin/
9 kubectl version --client
10 curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
11 install minikube-linux-amd64 /usr/local/bin/minikube
12 minikube version
13 cd /home/ubuntu
14 minikube start --driver=docker --force
15 kubectl cluster-info
16 kubectl get nodes
17 eval $(minikube docker-env)
18 touch Dockerfile pod.yml app.py
19 cat > Dockerfile
20 cat > app.py
21 cat > pod.yml
22 docker build -t ubuntu-hospital-web .
23 kubectl apply -f pod.yml
24 kubectl get pods
25 kubectl describe pod hospital-pod
26 kubectl logs hospital-pod
27 kubectl port-forward hospital-pod 5000:5000 --address=0.0.0.0
28 history
root@ip-172-31-31-44:/home/ubuntu#